

MANIPULATION DES COMMANDES ARTISAN

LARAVEL

A decorative graphic on the left side of the slide consisting of two parallel, wavy vertical lines. The inner line is a light blue color, and the outer line is white. They are set against a dark blue background.

INTRODUCTION

INTRODUCTION

Artisan est l'interface de ligne de commande incluse dans Laravel. Artisan existe à la racine de votre application en tant que script artisan et fournit un certain nombre de commandes utiles qui peuvent vous aider à construire votre application. Pour afficher une liste de toutes les commandes Artisan disponibles, vous pouvez utiliser la commande **list** :

```
php artisan list
```

INTRODUCTION

Chaque commande comprend également un écran d'"aide" qui affiche et décrit les arguments et les options disponibles de la commande. Pour afficher un écran d'aide, faites précéder le nom de la commande par **help** :

```
php artisan help migrate
```

TINKER (REPL)

Laravel Tinker est un puissant REPL pour le framework Laravel, alimenté par le paquet PsySH.

Toutes les applications Laravel incluent Tinker par défaut. Toutefois, vous pouvez installer Tinker à l'aide de Composer si vous l'avez préalablement supprimé de votre application :

```
composer require laravel/tinker
```

TINKER (REPL)

Tinker vous permet d'interagir avec l'ensemble de votre application Laravel en ligne de commande, y compris vos modèles Eloquent, vos travaux, vos événements, etc.

Pour entrer dans l'environnement Tinker, exécutez la commande tinker Artisan :

```
php artisan tinker
```

A decorative wavy line in light blue and white, resembling a stylized river or coastline, runs vertically along the left side of the image.

ÉCRITURE DES COMMANDES

ÉCRITURE DES COMMANDES

En plus des commandes fournies avec Artisan, vous pouvez créer vos propres commandes personnalisées. Les commandes sont généralement stockées dans le répertoire **app/Console/Commands** ; cependant, vous êtes libre de choisir votre propre emplacement de stockage tant que vos commandes peuvent être chargées par **Composer**.

GÉNÉRATION DE COMMANDES

Pour créer une nouvelle commande, vous pouvez utiliser la commande Artisan `make:command`. Cette commande va créer une nouvelle classe de commande dans le répertoire **app/Console/Commands**. Ne vous inquiétez pas si ce répertoire n'existe pas dans votre application - il sera créé la première fois que vous exécuterez la commande :

```
php artisan make:command SendEmails
```

STRUCTURE DE LA COMMANDE

Après avoir généré votre commande, vous devez définir des valeurs appropriées pour les propriétés **signature** et **description** de la classe. Ces propriétés seront utilisées lors de l'affichage de votre commande sur l'écran de synthèse.

La propriété **signature** vous permet également de définir les données d'entrée de votre commande.

La méthode **handle** sera appelée lorsque votre commande sera exécutée. Vous pouvez placer votre logique de commande dans cette méthode.

STRUCTURE DE LA COMMANDE

```
<?php
namespace App\Console\Commands;
use App\Models\User;
use App\Support\DripEmailer;
use Illuminate\Console\Command;
class SendEmails extends Command
{
    protected $signature = 'mail:send {user}';
    protected $description = 'Send a marketing email to a user';
    public function handle(DripEmailer $drip)
    {
        $drip->send(User::find($this->argument('user')));
    }
}
```

COMMANDES DE FERMETURE

Les commandes basées sur les fermetures offrent une alternative à la définition des commandes de la console en tant que classes. De la même manière que les fermetures de routes sont une alternative aux contrôleurs, pensez aux fermetures de commandes comme une alternative aux classes de commandes. Dans la méthode **commands** de **app/Console/Kernel.php**, Laravel charge le fichier **routes/console.php** :

```
protected function commands()  
{  
    require base_path('routes/console.php');  
}
```

COMMANDES DE FERMETURE

Même si ce fichier ne définit pas les routes HTTP, il définit les points d'entrée (routes) de la console dans votre application. Dans ce fichier, vous pouvez définir toutes vos commandes de console basées sur la fermeture en utilisant la méthode **Artisan::command**. La méthode **command** accepte deux arguments : la signature de la commande et une fermeture qui reçoit les arguments et les options de la commande :

```
Artisan::command('mail:send {user}', function ($user) {  
    $this->info("Sending email to: {$user}!");  
});
```

DESCRIPTIONS DES COMMANDES DE FERMETURE

Lorsque vous définissez une commande basée sur la fermeture, vous pouvez utiliser la méthode **purpose** pour ajouter une description à la commande. Cette description sera affichée lorsque vous exécuterez les commandes **php artisan list** ou **php artisan help** :

```
Artisan::command('mail:send {user}', function ($user) {  
    // ...  
})->purpose('Send a marketing email to a user');
```

A decorative wavy line in light blue and white on the left side of the image.

**DÉFINIR LES
DONNÉES
ATTENDÉES**

DÉFINIR LES DONNÉES ATTENDÉE

Lorsque l'on écrit des commandes en console, il est courant de recueillir des informations auprès de l'utilisateur par le biais d'arguments ou d'options. Avec Laravel, il est très pratique de définir les données que vous attendez de l'utilisateur en utilisant la propriété **signature** de vos commandes. La propriété **signature** vous permet de définir le nom, les arguments et les options de la commande dans une syntaxe unique, expressive et semblable à celle d'une route.

ARGUMENTS

Tous les arguments et options fournis par l'utilisateur sont entourés d'accolades. Dans l'exemple suivant, la commande définit un argument obligatoire : **user** :

```
protected $signature = 'mail:send {user}';
```

Vous pouvez également rendre les arguments facultatifs ou définir des valeurs par défaut pour les arguments :

```
// Argument facultatif...
```

```
'mail:send {user?}'
```

```
// Argument facultatif avec valeur par défaut...
```

```
'mail:send {user=foo}'
```

OPTIONS

Les options, comme les arguments, sont une autre forme de saisie de l'utilisateur. Les options sont préfixées par deux traits d'union (--) lorsqu'elles sont fournies par la ligne de commande. Il existe deux types d'options : celles qui reçoivent une valeur et celles qui n'en reçoivent pas. Les options qui ne reçoivent pas de valeur servent de "commutateur" booléen. Voyons un exemple de ce type d'option :

```
protected $signature = 'mail:send {user} {--queue}';
```

OPTIONS

Dans cet exemple, le commutateur **--queue** peut être spécifié lors de l'appel de la commande Artisan. Si le commutateur **--queue** est passé, la valeur de l'option sera **true**. Sinon, la valeur sera **false** :

```
php artisan mail:send 1 --queue
```

OPTIONS AVEC VALEURS

Ensuite, examinons une option qui attend une valeur. Si l'utilisateur doit spécifier une valeur pour une option, vous devez faire suivre le nom de l'option du signe = :

```
protected $signature = 'mail:send {user} {--queue=}';
```

OPTIONS AVEC VALEURS

Dans cet exemple, l'utilisateur peut transmettre une valeur pour l'option comme suit. Si l'option n'est pas spécifiée lors de l'appel de la commande, sa valeur sera nulle :

```
php artisan mail:send 1 --queue=default
```

OPTIONS AVEC VALEURS

Vous pouvez attribuer des valeurs par défaut aux options en spécifiant la valeur par défaut après le nom de l'option. Si aucune valeur d'option n'est passée par l'utilisateur, la valeur par défaut sera utilisée :

```
'mail:send {user} {--queue=default}'
```

RACCOURCIS DES OPTIONS

Pour attribuer un raccourci lors de la définition d'une option, vous pouvez le spécifier avant le nom de l'option et utiliser le caractère | comme délimiteur pour séparer le raccourci du nom complet de l'option :

```
'mail:send {user} {--Q|queue}'
```

Lorsque vous invoquez la commande dans votre terminal, les raccourcis d'options doivent être préfixés d'un seul trait d'union :

```
php artisan mail:send 1 -Q
```

TABLEAUX D'ENTRÉE

Si vous souhaitez définir des arguments ou des options qui attendent plusieurs valeurs d'entrée, vous pouvez utiliser le caractère *. Voyons d'abord un exemple qui spécifie un tel argument :

```
'mail:send {user*}'
```

Lors de l'appel de cette méthode, les arguments de l'utilisateur peuvent être passés dans l'ordre à la ligne de commande.

```
php artisan mail:send 1 2
```

TABLEAUX D'OPTIONS

Lorsque vous définissez une option qui attend plusieurs valeurs d'entrée, chaque valeur d'option transmise à la commande doit être précédée du nom de l'option :

```
'mail:send {--id=*}'
```

Une telle commande peut être invoquée en passant plusieurs arguments **--id** :

```
php artisan mail:send --id=1 --id=2
```

DESCRIPTIONS DES ENTRÉES

Vous pouvez attribuer des descriptions aux arguments d'entrée et aux options en séparant le nom de l'argument de la description à l'aide de deux points. Si vous avez besoin d'un peu plus d'espace pour définir votre commande, n'hésitez pas à répartir la définition sur plusieurs lignes :

```
protected $signature = 'mail:send  
    {user : The ID of the user}  
    {--queue : Whether the job should be queued}';
```

A decorative graphic on the left side of the slide, consisting of two parallel, wavy vertical lines. The inner line is a light blue color, and the outer line is white. They are set against a dark blue background.

COMMANDE I/O

RÉCUPÉRATION DES DONNÉES

Pendant l'exécution de votre commande, vous aurez probablement besoin d'accéder aux valeurs des arguments et des options acceptés par votre commande. Pour ce faire, vous pouvez utiliser les méthodes **argument** et **option**. Si un argument ou une option n'existe pas, **null** sera retourné :

```
public function handle()  
{  
    $userId = $this->argument('user');  
    //  
}
```

RÉCUPÉRATION DES DONNÉES

Si vous avez besoin de récupérer tous les arguments sous forme de tableau, appelez la méthode arguments :

```
$arguments = $this->arguments();
```

Les options peuvent être récupérées en utilisant les méthodes **option** et **options** :

```
// Récupérer une option spécifique...
```

```
$queueName = $this->option('queue');
```

```
// Récupérer toutes les options dans un tableau...
```

```
$options = $this->options();
```

INVITATION À LA SAISIE

En plus d'afficher la sortie, vous pouvez également demander à l'utilisateur de fournir des données pendant l'exécution de votre commande. La méthode **ask** demandera à l'utilisateur de répondre à la question posée, acceptera son entrée, puis la renverra à votre commande :

```
public function handle()  
{  
    $name = $this->ask('Quel est votre nom ?');  
}
```

INVITATION À LA SAISIE

La méthode **secret** est similaire à la méthode **ask**, mais la saisie de l'utilisateur ne sera pas visible pour lui lorsqu'il tape dans la console. Cette méthode est utile pour demander des informations sensibles telles que des mots de passe :

```
public function handle()  
{  
    $password = $this->secret('Quel est le mot de passe ?');  
}
```

DEMANDER UNE CONFIRMATION

Si vous devez demander à l'utilisateur une simple confirmation "oui ou non", vous pouvez utiliser la méthode **confirm**. Par défaut, cette méthode renvoie **false**. Toutefois, si l'utilisateur saisit **y** ou **yes** en réponse à l'invite, la méthode renvoie **true**.

```
if ($this->confirm('Souhaitez-vous continuer?')) {  
    //  
}
```

DEMANDER UNE CONFIRMATION

Si nécessaire, vous pouvez préciser que l'invite de confirmation doit renvoyer **true** par défaut en passant **true** comme deuxième argument à la méthode **confirm** :

```
if ($this->confirm('Souhaitez-vous continuer?', true)) {  
    //  
}
```

QUESTIONS À CHOIX MULTIPLES

Si vous devez donner à l'utilisateur un ensemble prédéfini de choix lorsqu'il pose une question, vous pouvez utiliser la méthode **choice**. Vous pouvez définir l'index du tableau de la valeur par défaut qui sera renvoyée si aucune option n'est choisie en passant l'index comme troisième argument de la méthode :

```
$name = $this->choice(  
'Quel est votre nom ?',  
['Taylor', 'Dayle'],  
$defaultIndex  
);
```

ÉCRITURE DE LA SORTIE

Pour envoyer une sortie à la console, vous pouvez utiliser les méthodes **line**, **info**, **comment**, **question**, **warn**, et **error**. Chacune de ces méthodes utilise les couleurs ANSI appropriées à son objectif.

```
public function handle()  
{  
    // ...  
  
    $this->info('The command was successful!');  
}
```

ÉCRITURE DE LA SORTIE

Pour envoyer une sortie à la console, vous pouvez utiliser les méthodes **line**, **info**, **comment**, **question**, **warn**, et **error**. Chacune de ces méthodes utilise les couleurs ANSI appropriées à son objectif.

```
public function handle()
{
    // ...
    $this->info('The command was successful!'); // Texte en vert
    $this->error('Something went wrong!'); // Texte en rouge
    $this->line('Display this on the screen'); // Texte plain
}
```

ÉCRITURE DE LA SORTIE

Pour envoyer une sortie à la console, vous pouvez utiliser les méthodes **line**, **info**, **comment**, **question**, **warn**, et **error**. Chacune de ces méthodes utilise les couleurs ANSI appropriées à son objectif.

```
public function handle()
{
    $this->info('The command was successful!'); // Texte en vert
    $this->error('Something went wrong!'); // Texte en rouge
    $this->line('Display this on the screen'); // Texte plain
    $this->newLine(); // Écrire une seule ligne vierge
}
```

A decorative wavy line in light blue and white, resembling a stylized wave or a calligraphic flourish, is positioned on the left side of the image.

EXÉCUTION PROGRAMMÉE DE COMMANDES

EXÉCUTION PROGRAMMÉE DE COMMANDES

Parfois, vous pouvez souhaiter exécuter une commande Artisan en dehors de l'interface CLI. Par exemple, vous pouvez souhaiter exécuter une commande Artisan à partir d'une route ou d'un contrôleur.

```
Route::post('/user/{user}/mail', function ($user) {  
    $exitCode = Artisan::call('mail:send', [  
        'user' => $user, '--queue' => 'default'  
    ]);  
});
```